Лабораторна робота №4

Знаходження алгебраїчних многочленів найкращого квадратичного наближення методом найменших квадратів.

Мета: Вивчити метод найменших квадратів знаходження алгебраїчних многочленів найкращого квадратичного наближення.

Основні відомості:

Нехай нам задана сітка $x_0, x_1, \ldots, x_n$ в вузлах якої відомі значення деякої функції f_i. Розглянемо методику побудови квадратичного наближення алгебраїчним многочленом за методом найменших квадратів. Шукатимемо апроксимуючий многочлен у вигляді

$$\varphi(x) = \sum_{j=0}^{m} a_j x^j .$$

Степінь многочлена m вибираємо меншим від загального числа вузлів сітки n. Коефіцієнти a_i знаходимо з умови мінімуму суми квадратів відхилень апроксимуючого многочлена у вузлах:

$$S = \sum_{i=0}^{n} \left(\varphi(x_i) - f_i\right)^2$$

Якщо відома точність значень функції у вузлах ε_i, то використовуючи поняття ваги

$$\rho_i = \frac{1}{\varepsilon_i} \sqrt{\frac{\varepsilon_0^2 + \varepsilon_1^2 + \ldots + \varepsilon_n^2}{n+1}}$$

можна отримати уточнену формулу:

$$S = \sum_{i=0}^{n} \rho_i \left(f_i - \varphi(x_i)\right)^2$$

В цьому випадку апроксимуюча функція буде ближче проходити до вузлів, в яких значення функції отримано точніше.

Для знаходження коефіцієнтів a_i,, обчислимо частинні похідні і прирівняємо їх до нуля:

$$\frac{\partial S}{\partial a_0} = 0 \ \ldots \ \frac{\partial S}{\partial a_i} = 0 \ \ldots \ \frac{\partial S}{\partial a_n} = 0$$

Отримаємо систему лінійних алгебраїчних рівнянь рівнянь:

$$\sum_{i=0}^{n} \rho_i \left(a_0 + a_1 x_i + \ldots + a_m x_i^m - f_i\right) = 0$$

$$\sum_{i=0}^{n} \rho_i \left(a_0 + a_1 x_i + \ldots + a_m x_i^m - f_i\right) x_i = 0$$

$$\sum_{i=0}^{n} \rho_i \left(a_0 + a_1 x_i + \ldots + a_m x_i^m - y_i \right) x_i^m = 0$$

Запишемо систему в компактнішій формі

$$b_{00} a_0 + b_{01} a_1 + \ldots + b_{0m} a_m = c_0$$
$$b_{10} a_0 + b_{11} a_1 + \ldots + b_{1m} a_m = c_1$$
$$\ldots$$
$$b_{m0} a_0 + b_{m1} a_1 + \ldots + b_{mm} a_m = c_m$$

Ми ввели скорочені позначення

$$b_{kl} = \sum_{i=0}^{n} \rho_i x_i^{k+l}$$

$$c_k = \sum_{i=0}^{n} \rho_i f_i x_i^k$$

В загальному виникає проблема оптимального вибору ступеня многочлена m. Як правило використовують наступну процедуру. Нехай величина ε визначає точність нашої апроксимації. Тоді для деякого значення m обчислюють дисперсію

$$\delta = \sqrt{\frac{\sum_{i=0}^{n} \left(\varphi(x_i) - f_i \right)^2}{n+1}}$$

Якщо $\delta \ll \varepsilon$, то збільшують m, якщо ж $\delta \gg \varepsilon$, то, навпаки, зменшують m. Якщо $\delta \approx \varepsilon$, то значення m є оптимальним. Якщо вийде, що $m_{опт} \approx n$, то необхідно використовувати інші методи для побудови апроксимаційного многочлена.

Для знаходження коефіцієнтів апроксимуючого алгебраїчного многочлена виникає задача розв'язку системи лінійних алгебраїчних рівнянь. Чисельні методи, які використовуються для розв'язку системи лінійних алгебраїчних рівнянь можна розділити на прямі і ітераційні методи. Серед прямих методів найбільш широко використовується метод Гауса і його модифікації. Розглянемо метод Гауса з вибором головного елемента. Суть методу полягає в приведенні системи рівнянь до системи рівнянь з трикутною матрицею, з якої потім послідовно знаходять значення всіх невідомих. Таким чином метод Гауса складається з двох етапів: прямого ходу і зворотного ходу. Для знаходження розв'язку необхідно виконати $n^3/3$ операцій. Розглянемо його особливості на прикладі системи лінійних алгебраїчних рівнянь з n невідомими:

$$a_{11} x_1 + a_{12} x_2 + \ldots + a_{1n} x_n = b_1,$$
$$\ldots$$
$$a_{i1} x_1 + a_{i2} x_2 + \ldots + a_{in} x_n = b_i,$$
$$\ldots$$
$$a_{n1} x_1 + a_{n2} x_2 + \ldots + a_{nn} x_n = b_n.$$

яка може бути представлена у матричній формі $AX = B$.

Позначимо через U верхню трикутну матрицю, яку ми отримаємо після прямого ходу методу Гауса, а через f - перетворений вектор вільних членів.

Найпростіше програмно реалізувати вибір головного елемента в стовпчику. Вибір найбільшого по модулю головного елементу дає змогу мінімізувати похибку округлень.

Приймемо, що всі елементи матриці U рівні нулю $u_{ij} = 0$.

Розглянемо перший стовпчик. Нехай a_{i1} – найбільший за модулем його елемент. Виконаємо перестановку 1 і i рядків місцями.

$prom = a_{1j}$,

$a_{1j} = a_{ij}$, $\quad (j = 1, n)$

$a_{ij} = prom$

Далі з допомогою елемента a_{11} виключають решту елементів першого стовпчика починаючи з другого і до n:

$u_{1j} = a_{1j} \quad (j = 1, \ldots, n)$

$f_1 = b_1$

$a_{ij}^{(1)} = a_{ij} - a_{1j} \dfrac{a_{i1}}{a_{11}} \quad \begin{pmatrix} i = 2, \ldots, n \\ j = 2, \ldots, n \end{pmatrix}$

$b_i^{(1)} = b_i - b_1 \dfrac{a_{i1}}{a_{11}} \quad (i = 2, \ldots, n)$

де u_{ij}, f_i - елементи перетвореної до трикутного вигляду матриці. $a_{ij}^{(k)}$ - елементи вихідної матриці після k - перетворення.

Розглянемо далі другий стовпчик. Відкидаємо перший елемент u_{12} і серед усіх $a_{i2}^{(1)}$ $(i = 2 \ldots n)$ вибираємо найбільший за модулем. Перестановкою рядків розміщаємо його в другому рядку. З його допомогою виключаємо всі інші елементи другого стовпчика.

$u_{2i} = a_{2i}^{(1)} \quad (i = 2 \ldots n)$

$f_2 = b_2^{(1)}$

$a_{ij}^{(2)} = a_{ij}^{(1)} - a_{2j}^{(1)} \dfrac{a_{i2}^{(1)}}{a_{22}^{(1)}} \quad \begin{pmatrix} i = 3 \ldots n \\ j = 3 \ldots n \end{pmatrix}$

$b_i^{(2)} = b_i^{(1)} - b_2^{(1)} \dfrac{a_{i2}^{(1)}}{a_{22}^{(1)}} \quad (i = 3 \ldots n)$

Розглянемо *k*-й крок. В *k*-му стовпчику серед усіх елементів $a_{ik}^{(k-1)}$ $(i = k \ldots n)$ вибираємо найбільший за модулем. Перестановкою рядків переміщаємо його в k -й рядок.

$u_{ki} = a_{ki}^{(k-1)} \quad (i = k, n)$

$f_k = b_k^{(k-1)}$

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - a_{kj}^{(k-1)} \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} \qquad (i, j = k+1, n)$$

$$b_i^{(k)} = b_i^{(k-1)} - b_k^{(k-1)} \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}} \qquad (i = k+1, n)$$

В результаті ми отримаємо систему рівнянь $U \cdot X = f$, де матриця U має трикутний вигляд. Описана процедура називається прямим ходом методу Гауса. На зворотному ході методу Гауса можна знайти значення невідомих x:

$$\begin{cases} x_n = \dfrac{f_n}{u_{nn}} \\ x_{n-1} = \dfrac{1}{u_{n-1,n-1}} \left(f_{n-1} - u_{n-1,n} x_n \right) \\ x_i = \dfrac{1}{u_{ii}} \left(f_i - \sum\limits_{j=i+1}^{n} u_{ij} x_j \right) \end{cases}$$

Хід роботи.

1. Скласти програму для табуляції заданих даних про температуру на відрізку $[x_0, x_n]$ з кроком $h = \frac{x_n - x_0}{n}$, де n - 24 місяці задане число вузлів. Отриману сукупність вузлів $\{x_i, y_i\}$ використати в подальшому як набір вхідних даних.
2. Скласти програму, яка для заданої табличної функції $\{x_i, f_i\}$ методом найменших квадратів знаходить апроксимуючий многочлен степені m. В програмі записати наступні функції: зчитування вхідних даних з текстового файлу – масивів $x_i, f_i, i = 0 \ldots n$, функції формування масивів B та C, функцію розв'язування системи лінійних алгебраїчних рівнянь методом Гауса з вибором головного елементу по стовпцях, функцію обчислення алгебраїчного многочлена найкращого квадратичного наближення по відомому масиву коефіцієнтів A, функцію обчислення похибки $\varepsilon(x) = |f(x) - \varphi(x)|$.
3. Знайти апроксимуючий многочлен і обчислити дисперсію для випадків $m = 1, \ldots, 10$. Побудувати графік залежності величини дисперсії від степені апроксимуючого многочлена. Вибрати оптимальне значення степені m виходячи з умови мінімуму дисперсії.

4. Протабулювати на відрізку $[x_0, x_n]$ з кроком $h_1 = \frac{x_n - x_0}{20n}$ похибку для випадків $m = 1,\ldots,10$. Побудувати графіки цих функцій.
5. Результати виконання лабораторної роботи оформити у вигляді звіту.

Розроблена програма повинна:

1. Зчитувати середньомісячні температури з CSV.
2. Будувати квадратичний або вищий поліном методом найменших квадратів.
3. Обчислювати дисперсію для різних степенів полінома і вибирає оптимальний.
4. Створювати графік апроксимації та фактичних даних.
5. Табулювати і будувати графік похибки апроксимації.
6. Екстраполювати прогноз температури на наступні 3 місяці.

Приклад даних у файлі csv

```
Month,Temp
1,-2
2,0
3,5
4,10
5,15
6,20
7,23
8,22
9,17
10,10
11,5
12,0
13,-10
14,3
15,7
16,13
17,19
18,20
19,22
20,21
21,18
22,15
23,10
24,3
```

Псевдокод:

```
# ------------------------------
# 1. Вхідні дані
# ------------------------------
x = [1,2,...,24]        # місяці
y = [-2,0,5,10,15,20,...,3]   # температури

# ------------------------------
# 2. Функції МНК
# ------------------------------
function form_matrix(x, m):
    A = нульова матриця розміром (m+1, m+1)
    for i = 0 to m:
        for j = 0 to m:
            A[i,j] = сума(x[k]^(i+j)) по всіх k
    return A

function form_vector(x, y, m):
    b = нульовий вектор розміром (m+1)
    for i = 0 to m:
        b[i] = сума(y[k] * x[k]^i) по всіх k
    return b

function gauss_solve(A, b):
    # Прямий хід з вибором головного елемента
    for k = 0 to n-1:
        знайти рядок max_row з найбільшим |A[i,k]|
        поміняти рядки k та max_row місцями у A та b
        for i = k+1 to n-1:
            factor = A[i,k] / A[k,k]
            A[i,k:] = A[i,k:] - factor * A[k,k:]
            b[i] = b[i] - factor * b[k]
    # Зворотній хід
    x_sol = нульовий вектор розміром n
    for i = n-1 downto 0:
        x_sol[i] = (b[i] - сума(A[i,j]*x_sol[j] для j=i+1..n-1)) / A[i,i]
    return x_sol

function polynomial(x, coef):
    y_poly = нульовий вектор того ж розміру, що x
    for i = 0 to length(coef)-1:
        y_poly += coef[i] * x^i
    return y_poly

function variance(y_true, y_approx):
```

```
    return середнє((y_true - y_approx)^2)

# ------------------------------
# 3. Вибір оптимального ступеня полінома
# ------------------------------
max_degree = 4
for m = 1 to max_degree:
    A = form_matrix(x, m)
    b_vec = form_vector(x, y, m)
    coef = gauss_solve(A, b_vec)
    y_approx = polynomial(x, coef)
    var = variance(y, y_approx)
    зберегти var у список variances

optimal_m = індекс мінімального значення у variances + 1

# ------------------------------
# 4. Побудова апроксимації
# ------------------------------
A = form_matrix(x, optimal_m)
b_vec = form_vector(x, y, optimal_m)
coef = gauss_solve(A, b_vec)
y_approx = polynomial(x, coef)

# ------------------------------
# 5. Прогноз на наступні 3 місяці
# ------------------------------
x_future = [25,26,27]
y_future = polynomial(x_future, coef)

# ------------------------------
# 6. Похибка апроксимації
# ------------------------------
error = y - y_approx

# ------------------------------
# 7. Вивід результатів
# ------------------------------
друкувати дисперсії для різних ступенів
друкувати оптимальний ступінь
друкувати прогноз y_future
побудувати графіки: фактичні дані, апроксимація, похибка
```